

Chapter 5

Tasks

5.4 Typical Task Operations

- Kernel must provide *task-management services*
 - The actions that a kernel performs to support task
 - Create and maintain the TCB and task stacks
 - The API that a kernel provides for developer to manipulate tasks
 - Create and delete tasks
 - Control task scheduling
 - Obtain task information

Task Creation and Deletion

- Two mechanisms to create a task
 - Create a task first and then start it
 - Debugging
 - When special initialization needs to occur between the times that a task is created and started
 - Create and start a task using one kernel call

Operation	Description
Create	Creates a task
Delete	Deletes a task

Task Creation and Deletion (Cont.)

- User-configurable hooks
 - Execute programmer-supplied functions at the time of specific kernel events
 - Programmer must first register the function with the kernel
 - Pass a function pointer to a kernel-provided API
 - Kernel executes this function when
 - A task is first created or deleted
 - A task is suspended for any reason and a context switch occurs

Delete a Task

- Many kernel allow any task to delete any other task
 - Kernel terminates the task and free memory by deleting task's TCB and stack
- However, when tasks executes, they can acquire memory or access resources
 - Memory or resource leaks would occur if the task is deleted incorrectly

Memory and Resource Leak

- ❑ Memory leak occurs when memory is acquired but not released
 - Cause the system to run out of memory eventually
- ❑ Resource leak occurs when a resource is acquired but never released
 - Result in a memory leak because each resource takes up space in memory
- ❑ Task-deletion locks
 - A pair of calls that protect a task from being prematurely deleted during a critical section of code

Delete a Task (Cont.)

- ❑ For example, delete a task that still holds a semaphore token would result in
 - A corrupt data structure, due to an incomplete write operation
 - An unreleased semaphore, which will not be available for other tasks that might need to acquire it
 - An inaccessible data structure, due to the unreleased semaphore

Task Scheduling

- Manual scheduling
 - Many kernels provides a set of API calls that allow developers to control when a task moves to a different state

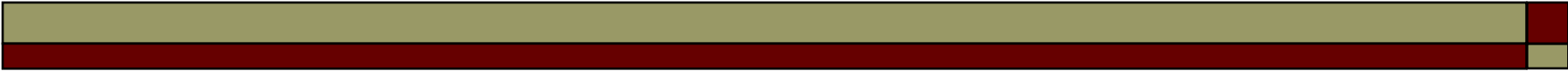
Operation	Description
Suspend	Suspends a task
Resume	Resumes a task
Delay	Delays a task
Restart	Restarts a task
Get Priority	Gets the current task's priority
Set Priority	Dynamically sets a task's priority
Preemption lock	Locks out higher priority tasks from preempting the current task
Preemption unlock	Unlocks a preemption lock

Task Scheduling (Cont.)

- Suspend and Resume
 - For debugging purposes
 - Suspend a high-priority task so that lower priority task can execute
- Delay a task
 - Allow manual scheduling
 - Wait for an external condition that does not have an associated interrupt
 - Polling: wait up after a set time to check a specified condition or event had occurred

Task Scheduling (Cont.)

- Restart
 - Begin the task as if it had not been previously executing
 - Useful during debugging or when initializing a task after a catastrophic error
- Get and Set Priority
 - Control task scheduling manually
 - Helpful during a priority inversion



Task Scheduling (Cont.)

- Preemption lock
 - A pairs of calls used to disable and enable preemption in applications
 - Useful if a task is in a critical section of code

Obtaining Task Information

- Useful for debugging and monitoring
- Get ID: obtain a task's ID
- Get TCB: obtain a task's TCB by its ID
 - Only a snapshot of the task context

5.5 Typical Task Structure

- Task are structured in one of two ways
 - Run to completion
 - Useful for initialization and startup
 - Endless loop
 - Do the majority of the work in the application by handling inputs and outputs

Run-to-Completion Tasks

- Application initialization task typically has a higher priority than the application tasks it creates
 - Its initialization work is not preempted
- Application initialization task suspends or deletes itself after it completes its work
 - So the newly created task can run

Pseudo Code for a Run-to-Completion Task

```
RunToCompletionTask ()  
{  
    Initialize application  
    Create 'endless loop tasks'  
    Create kernel objects  
    Delete or suspend this task  
}
```

Endless-Loop Tasks

- Critical part of the design of an endless-loop
 - One of more blocking calls within the body of the loop
 - Allow lower priority tasks to run

```
EndlessLoopTask ()
{
    Initialization code
    Loop Forever
    {
        Body of loop
        Make one or more blocking calls
    }
}
```


5.6 Synchronization, Communication, and Concurrency

- Intertask primitives
 - Use in task synchronization and communicate amongst themselves
 - Kernel objects facilitates synchronization and communication between two or more threads of executions
 - Semaphore, message queues, signals, and pipes
 - Discussed in detail in later chapter